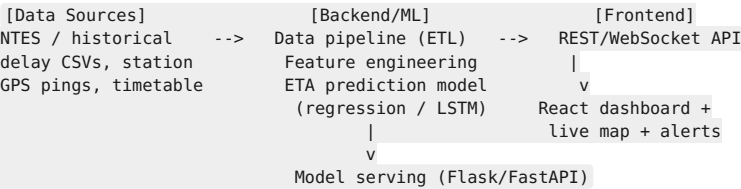


SIH26028 - Dynamic ETA Forecast - Team Structure & Work Division

SIH26028 — Dynamic ETA Forecast for Coaching Trains

Team Structure, Architecture & Work Division (6 Members)

1. Project Architecture (High Level)



Core modules to build: 1. Data collection & cleaning pipeline 2. Feature engineering (delay history, section congestion, weather, day-of-week, halted time) 3. ML model (baseline: gradient boosting → stretch: LSTM/time-series) 4. Backend API serving predictions 5. Frontend dashboard (live map, station-wise ETA, delay-reason explainability) 6. Presentation/demo narrative + pitch deck

2. Team Roles & Work Division (6 Members)

#	Role	Primary Responsibility	Key Deliverables
1	Team Lead / ML Engineer	Owns the prediction model end-to-end	Baseline model (Day 1), tuned model (Day 2), accuracy benchmarking vs. static timetable
2	Data Engineer	Sources, scrapes, and cleans data	Historical delay dataset, NTES/API scraper or synthetic data generator, feature tables
3	Backend Developer	Builds API layer connecting model to frontend	FastAPI/Flask endpoints, model-serving pipeline, WebSocket for live updates
4	Frontend Developer (Dashboard/Map)	Builds the visual live-tracking UI	Interactive map (train position + ETA per station), delay-reason panel
5	Frontend Developer (UX/Product)	Builds passenger-facing views + polish	Mobile-responsive passenger view, notifications/alerts UI, overall design consistency
6	Research & Presentation Lead	Domain research, validation, and pitch	Problem-statement alignment doc, judge-facing PPT, demo script, competitor analysis (RailYatri, NTES, Where Is My Train)

Note: In a 6-person hackathon team, roles overlap in practice — pair people up (e.g., ML Engineer + Data Engineer collaborate constantly; the two frontend devs split map vs. passenger views but share a component library). Treat this as primary ownership, not isolation.

3. Skill-to-Role Mapping (fill in based on your team)

Member	Strongest Skill	Suggested Role
_____	Python / ML / stats	ML Engineer
_____	SQL / scripting / APIs	Data Engineer
_____	Node/Python backend, system design	Backend Developer
_____	React, maps (Leaflet/Mapbox), data viz	Frontend – Dashboard
_____	React/Figma, product sense	Frontend – UX
_____	Writing, storytelling, research	Research & Presentation

4. Timeline (36-Hour Hackathon Structure)

Phase	Time	Focus	Who's active
Kickoff & Design	Hr 0-3	Finalize scope, data source confirmed, architecture locked, wireframes sketched	All 6
Parallel Build I	Hr 3-12	Data pipeline + baseline model; API skeleton; UI skeleton	Data Eng + ML (data/model), Backend + Frontend×2 (skeleton)
Integration Checkpoint 1	Hr 12-14	Connect model output → API → frontend (even with dummy predictions)	Backend + both Frontend
Parallel Build II	Hr 14-24	Model tuning/accuracy improvement; full UI features; alerts/notifications	ML + Data Eng, Frontend×2
Integration Checkpoint 2	Hr 24-28	Full pipeline live end-to-end; bug fixing; edge cases (train not found, no data)	All 6
Polish & Rehearsal	Hr 28-33	UI polish, demo data curation, pitch deck, rehearse live demo	Presentation Lead drives, all support
Buffer / Sleep rotation	Hr 33-36	Final testing, backup demo video (in case live	All 6

		demo fails), submission	
--	--	-------------------------	--

Pro tip: Record a backup demo video by Hour 30 — live demos with real-time data feeds are the #1 failure point in judging rounds.

5. Resource & Tooling Plan

Need	Recommended Tool/Resource	Notes
Historical train delay data	NTES scrape / Kaggle Indian Railways datasets / RailRadar-style APIs	If live API access is blocked, use historical CSVs + simulate “live” replay for demo
ML framework	scikit-learn (baseline: XGBoost/LightGBm) → PyTorch/Keras if time permits (LSTM)	Don’t over-engineer — a well-tuned gradient boosting model often beats LSTM on limited hackathon data
Backend	FastAPI (Python) — pairs naturally with your ML stack	Avoids context-switching between Python ML and a separate backend language
Frontend map	Leaflet.js or Mapbox GL (free tier)	Leaflet is simpler to get running fast
Hosting for demo	Render/Railway (backend) + Vercel/Netlify (frontend)	Free tiers are enough for a hackathon demo
Version control	GitHub with a shared repo, branch-per-feature	Set this up in the first hour — don’t skip it
Design	Figma (even a rough wireframe before coding saves rework)	Frontend UX lead owns this

6. Risk Register (things likely to bite you)

Risk	Mitigation
No access to true live train GPS data	Use historical data + simulate “live progression” for the demo; be upfront about this limitation to judges — it’s expected at this stage
Model accuracy looks weak vs. static timetable	Benchmark clearly: even a modest 15–20% MAPE improvement over static timetable (like published research shows) is a strong, defensible result — don’t chase perfection
Frontend/backend integration breaks late	Do 2 integration checkpoints (Hr 12–14 and Hr 24–28), not just one at the end
Team burns out / uneven workload	Rotate a “float” person each 6-hour block to help whichever module is behind
Judges ask “how is this different from RailYatri/NTES?”	Have this answer ready <i>before</i> the demo — see differentiation angle below

7. Suggested Differentiation Angle (for your pitch)

Since apps like RailYatri/Where Is My Train already do ETA prediction, your pitch should lead with **one clear edge**, e.g.: - **Explainability**: showing *why* a train is delayed (upstream congestion, halted time, historical pattern) — not just a number. - **Station-level granularity**: predicting ETA at every intermediate station, not just the next major stop. - **Cascading-delay awareness**: factoring in shared-section congestion from other trains (even a simplified version is a differentiator).

Pick one and build the demo narrative around it — don’t try to claim all three.